



Curso de Lenguaje C - Documentación del Hacker

¡Bienvenido a tu manual de referencia, pisha! Aquí tienes resumido todo lo que hemos montado y aprendido hasta ahora.



1. Entorno de Desarrollo (Build System)

Hemos configurado un sistema profesional en VS Code que imita a los grandes IDEs like Code::Blocks.

Estructura de Carpetas

Cada proyecto debe tener esta estructura:

- **Proyecto/**
 - **main.c**: Tu código fuente.
 - **.vscode/tasks.json**: El "molde" de compilación.
 - **obj/**: Aquí van los archivos **.o** (código máquina intermedio).
 - **bin/Debug/**: Aquí va el delicioso **.exe** final.

Cómo Compilar

1. Abre el archivo **.c** que quieras compilar.
 2. Pulsa **Ctrl + Shift + B**.
 3. Elige **"Build C (Professional Structure)"**.
 4. Ejecuta el programa desde la terminal: **./bin/Debug/main.exe**.
 5. Sube tus cambios: **git push origin main**.
-



2. Conceptos de C Aprendidos

Variables y Tipos

- **int**: Números enteros.
- **float** / **double**: Números con decimales (usa **float n = 5.0f**).
- **char**: Un solo carácter ('A').
- **long long**: Números astronómicos.
- **unsigned**: Números solo positivos (llegan más alto).

Entrada y Salida (I/O)

La comunicación principal con el programa se hace mediante la librería **<stdio.h>**.

Impresión Avanzada (**printf**):

- **%d**: Imprime enteros (**int**).
- **%f**: Imprime decimales (**float**). **Truco**: Usa **%.2f** para redondear a 2 decimales.
- **%c**: Imprime caracteres (**char**).
- **%s**: Imprime una cadena de texto (String).

- **Padding:** Usa `%04d` para forzar que el número ocupe siempre 4 espacios rellenando con ceros a la izquierda (ej: 7 se imprime como 0007).

Lectura Avanzada (`scanf`):

- El ampersand `&` es obligatorio para números o `char`, ya que necesita saber la DIRECCIÓN donde guardar el dato. (Con strings `%s` no hace falta `&` porque ya son punteros).
- **Lectura Múltiple:** Puedes pedir varios datos a la vez con un formato estricto. Ej: `scanf("%d-%d", &dia, &mes)` solo funcionará si el usuario escribe literalmente "01-12".
- **Limitador de Desbordamiento:** Usa `%5s` para obligar a `scanf` a leer solo 5 letras de golpe, previniendo que el usuario crashee tu programa si mete un texto infinito.

Control de Flujo

- **Condicionales (`if` / `else`):** Para tomar decisiones.
- **Bucle `while`:** Repite "mientras" se cumpla una condición.
- **Bucle `for`:** Repite un número fijo de veces (ideal para tablas o conteos).

3. Trucos Pro de Hacker

El Parche de Caracteres Especiales (Windows)

Para que los acentos y la `¡` salgan bien, añade esto al principio de tu `main`:

```
#include <windows.h>
// ... dentro de main ...
SetConsoleOutputCP(CP_UTF8);
SetConsoleCP(CP_UTF8);
```

El Fantasma del Buffer

Si el programa se cierra solo, usa dos `getchar()` al final. El primero se come el "Enter" residual del teclado y el segundo pausa la consola.

El Archivo `.o` (Objeto)

Es código máquina ya traducido pero sin "enlazar" con las bibliotecas externas. El Linker lo coge y crea el `.exe`.

El Bucle Infinito de `scanf` (Protección de Tipos)

Si pides un número (`%d` o `%f`) y el usuario mete una letra, `scanf` se bloquea, no limpia el buffer y crea un bucle infinito espantoso. **La solución Hacker:** Comprobar si `scanf` ha leído con éxito su objetivo (`!= 1`). Si falla, "tiramos de la cadena" del buffer con `getchar()` hasta encontrar el Intro (`\n`).

```
if (scanf("%d", &numero) != 1) {
    while (getchar() != '\n'); // Limpia la basura introducida
```

```
    printf("¡Error! Mete un número real.\n");  
}
```

4. Reto del Hacker: Aleatoriedad

Para crear juegos o programas impredecibles, usamos la generación de números aleatorios.

Conceptos Clave

- **#include <time.h>**: Necesaria para obtener la hora del sistema.
- **srand(time(NULL))**: Inicializa la "semilla". Hazlo una sola vez al principio del **main** para que los números cambien en cada partida.
- **rand() % 100 + 1**:
 - **rand()**: Genera un número enorme al azar.
 - **% 100**: Limita el número entre 0 y 99.
 - **+ 1**: Ajusta el rango para que sea de 1 a 100.

5. Arrays (Arreglos / Listas)

Los arrays permiten guardar múltiples valores del mismo tipo bajo un solo nombre.

Puntos clave

- **Declaración**: **int notas[5]**; reserva espacio para 5 números.
- **Índices**: Empezamos siempre en **0**. Las posiciones serían: **0, 1, 2, 3, 4**.
- **Acceso**: **notas[0] = 10**; guarda un 10 en la primera posición.
- **Bucle FOR**: Es la herramienta perfecta para recorrer arrays (pedir datos o imprimirlos).

Encontrar Máximo y Mínimo

Para encontrar el valor más grande o pequeño en una lista:

1. **Inicialización**: Asume que el primer elemento (**lista[0]**) es el máximo y el mínimo.
2. **Comparación**: Recorre el resto del array con un bucle.
3. **Actualización**: Si encuentras un número mayor que tu "máximo", actualízalo. Haz lo mismo para el mínimo.

Cálculo de la Media

Para calcular la media de un array:

1. **Acumular totales**: Necesitamos una variable que sume los datos dentro del bucle (**suma += lista[i]**).
2. **División flotante**: Ojo con esto, al dividir enteros, fuérzalos a flotante: **(float)suma / total_elementos**.

6. Cadenas de Texto (Strings)

En C no existe un tipo de dato "String" mágico. Lo que tenemos son **arrays de caracteres (char)**.

Puntos Clave

- **Declaración:** `char mensaje[100];` reserva hasta 99 letras y el nulo final (`\0`) que marca el final.
- **Lectura segura:** Para leer texto con espacios sin petar la memoria, usamos `fgets(mensaje, tamaño, stdin)`.
- **Librería `<string.h>`:** Una librería brutal que nos da funciones guapas como `strlen(mensaje)` para conseguir la longitud exacta de la cadena introducida.

Truco Hacker con fgets

`fgets` tiene un "defecto": ¡se guarda también el salto de línea al pulsar Enter (`\n`)! Así se lo extirpamos:

```
int longitud = (int)strlen(mensaje);
// Si la última letra es el intro, lo machacamos con el nulo final
if (longitud > 0 && mensaje[longitud - 1] == '\n') {
    mensaje[longitud - 1] = '\0';
}
```

Cifrado de Caracteres (Letras como Números)

Las letras en C internamente son números (código ASCII). Esto nos permite jugar con ellas. Para hacer cifrados (como el de César):

- `mensaje[i] + desplazamiento` para cambiar la letra.
- Usar `% 26` (las letras del alfabeto) para que al pasar de la 'Z', vuelva a la 'A'.

🔗 7. Funciones (Modularidad)

Las funciones nos permiten dividir un programa enorme en piezas más pequeñas, reutilizables y fáciles de entender ("Divide y vencerás").

Anatomía de una Función

1. **Prototipos (Arriba del `main`):** Avisamos al compilador de qué funciones existen. ¡Es como el índice de un libro!

```
float sumar(float a, float b);
```

2. **Llamada (Dentro del `main`):** Ejecutamos la función cuando nos convenga, pasándole la "materia prima" (argumentos).

```
resultado = sumar(5.0, 3.0);
```

3. **Definición (Abajo del `main`):** El código real que dice qué hace la función.

```
float sumar(float a, float b) {  
    return a + b;  
}
```

Tipos de Funciones

- **void:** Funciones "mudas". Ejecutan acciones (como imprimir un menú) pero no te devuelven cálculos. Ej: `void mostrar_menu(void)`.
- **int, float, etc.:** Operan y **RETORNAN** un valor de vuelta al `main` usando la palabra clave `return`.

Scope (Ámbito de las Variables)

👑 **Regla de Oro Hacker:** Las variables que creas dentro de una función **NACEN** y **MUEREN** en esa función. El `num1` de tu `main` no es el mismo `a` de tu función `sumar()`, simplemente **se le copia el valor**. A esto se le llama **"Paso por Valor"**.

👑 8. Punteros (La Magia y El Alma de C)

Los punteros son la razón por la que C es un lenguaje tan rápido y de tan bajo nivel. Un puntero no guarda un número o una letra, **guarda una dirección física de la memoria RAM**.

Direcciones de Memoria (&)

Toda variable (`int saldo = 100;`) vive en una "caja" dentro de la RAM.

- Si imprimes `saldo`, ves `100`.
- Si imprimes `&saldo`, ves la ubicación real de la caja (ej: `0x00A1F0`).

Punteros (*)

Podemos crear variables diseñadas expresamente para guardar esas ubicaciones (direcciones).

```
int saldo = 100;  
int *puntero_saldo = &saldo; // Guarda la dirección de saldo
```

Paso por Referencia (El Juego del Trilero)

En el tema de Funciones vimos que el "Paso por Valor" solo enviaba copias. Con los punteros, podemos enviar **la ubicación real** de la variable a otra función para que esta la modifique a distancia.

```
// La función recibe una llave (dirección)  
void hackear_cuenta(int *cuenta_bancaria) {  
    // Con el asterisco abrimos la caja a distancia y le metemos pasta nueva  
    *cuenta_bancaria = 999999;  
}
```

```

}

int main() {
    int mi_saldo = 0;
    // Le pasamos la dirección de nuestra variable usando el ampersand (&)
    hackear_cuenta(&mi_saldo);
    // mi_saldo ahora vale 999999
}

### 🏠 El Secreto Mejor Guardado: Arrays = Punteros

En C, **el nombre de un Array es en sí mismo un Puntero** que apunta a su primer elemento (posición `0`).
```c
int notas[3] = {5, 7, 9};
int *puntero = notas; // No hace falta poner ¬as, porque "notas" ya es la dirección

// notas[0] es la misma dirección que "puntero"

```

## Aritmética de Punteros (Saltando por la RAM)

Como los elementos del array están organizados secuencialmente (uno detrás de otro) en la memoria, podemos usar simples sumas matemáticas para avanzar a la siguiente posición de memoria a nivel de hardware.

```

printf("%d", *puntero); // Imprime la pos 0: 5
printf("%d", *(puntero + 1)); // Avanza 1 bloque y lee la pos 1: 7
printf("%d", *(puntero + 2)); // Avanza 2 bloques y lee la pos 2: 9

```

## Cadenas de Texto y Punteros (`char *`)

Dado que un String en C no es más que un Array de caracteres (`char[]`), cuando pasamos un texto a una función, lo que recibe la función es puramente un puntero (`char *`). Esto significa que **cualquier cambio que hagas en la función, modificará el texto original**.

Para recorrer un texto usando punteros, buscamos el carácter nulo (`\0`) que indica el final de la cadena:

```

void a_mayusculas(char *texto) {
 while (*texto != '\0') {
 *texto = *texto - 32; // Modificamos la letra actual
 texto++; // Avanzamos el puntero a la siguiente letra
 }
}

```

## Memoria Dinámica (`malloc` y `free`)

Habitualmente guardamos los datos especificando el tamaño antes de compilar (`int array[5]`). Pero, ¿qué pasa si el usuario es quien te dice cuánta memoria necesita mientras ejecuta el programa? Para eso está la **Memoria Dinámica** en la librería `<stdlib.h>`.

1. **Pedir RAM (`malloc`)**: Le pedimos espacio al Sistema Operativo pasándole la cantidad de bytes que necesitamos. El SO nos devuelve el "puntero" hacia esa nueva memoria.
2. **Liberar RAM (`free`)**: ¡CRÍTICO! Toda la RAM pedida con `malloc` se queda reservada eternamente. Si tu programa termina y no devuelves la RAM usando `free(puntero)`, habrás generado una "fuga de memoria" (Memory Leak).

```
#include <stdlib.h> // Para malloc y free

int cant;
scanf("%d", &cant);

// Pedimos hueco para 'cant' elementos enteros
int *bots = (int *)malloc(cant * sizeof(int));

// ... Usamos el array normalmente: bots[0] = 500 ...

// IMPORTANTE: Devolvemos la RAM al SO
free(bots);
```

### Resumen Hacker Completo:

- Usa `&` para conseguir la DIRECCIÓN de una variable real.
- Usa `*` para ABRIR la caja de una dirección y modificar su contenido.
- Las Cadenas y Arrays son en realidad Punteros encubiertos.
- `array[i]` es internamente equivalente a desplazarse por la RAM: `*(array + i)`.
- Si pides RAM on-the-fly (`malloc`), debes devolverla (`free`) o tu PC colapsará.

---

## 9. Estructuras (`structs` y `typedef`)

Hasta ahora creábamos variables para un solo dato (un número, una letra). Con las Estructuras podemos crear **nuestros propios tipos de datos complejos**, agrupando varias variables bajo un mismo nombre.

Imagina crear el "Molde" para la Ficha de un Jugador RPG:

```
// Definimos el molde general
struct Personaje {
 char nombre[50];
 int nivel;
 float vida_maxima;
};
```

El Truco Hacker: `typedef`

Para no tener que escribir `struct Personaje mi_heroe;` todo el tiempo, le ponemos un Alias usando `typedef`, convirtiéndolo oficialmente en un tipo de dato nuevo (como si fuera un simple `int`).

```
// Ahora "Heroe" es el nuevo tipo de dato
typedef struct Personaje Heroe;

// Declaramos a los héroes usando nuestro molde
Heroe p1;
Heroe p2;
```

## Accediendo a los Datos (El Punto .)

Para leer o modificar las variables que viven *dentro* de nuestro Struct, usamos el operador punto `.`:

```
p1.nivel = 99;
p1.vida_maxima = 1500.5f;

// Ojo: Las Strings internas no se pueden reasignar con =, hay que usar strcpy()
strcpy(p1.nombre, "Lord Viernes");
```

## Punteros y Structs (El Operador Flecha ->)

¿Qué pasa si queremos pasar nuestro `Heroe` a una función para curarlo? Si lo pasamos normal (Por Valor), C copiará **todos** los datos de la ficha en la RAM para enviárselo a la función. Si el Struct es gigante, tu juego irá a tirones.

**La solución hacker:** Pasamos un "Puntero al Struct" (Paso por Referencia). Así solo enviamos una ligera dirección de memoria.

El único problema es que **con los punteros a structs, el punto (.) ya no funciona**. Tienes que usar el **Operador Flecha (->)** para "viajar" por el puntero y acceder al dato.

```
// La función recibe la DIRECCIÓN de un héroe
void curar_heroe(Heroe *pj) {
 // Usamos la flecha en lugar del punto
 pj->vida_maxima += 50.0f;
}

int main() {
 Heroe p1;
 // ... datos de p1 ...

 // Le pasamos la dirección de memoria (&) de nuestro héroe
 curar_heroe(&p1);
}
```



¡A seguir dándole caña al código! 🚀 👁